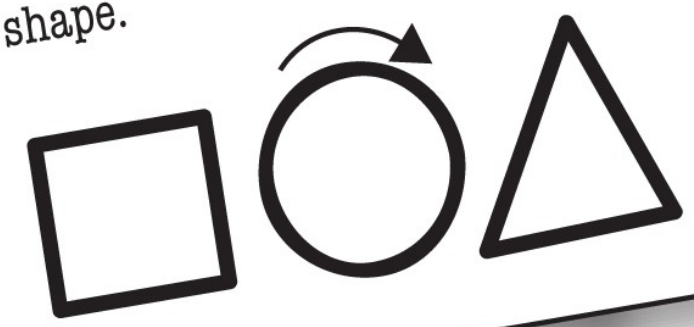

We're going to Objectville! We're leaving this dusty ol' procedural town for good. I'll send you a postcard.

There will be shapes on a GUI, a square, a circle, and a triangle. When the user clicks on a shape, the shape will rotate clockwise 360º (i.e. all the way around) and play an AIF sound file specific to that particular shape.



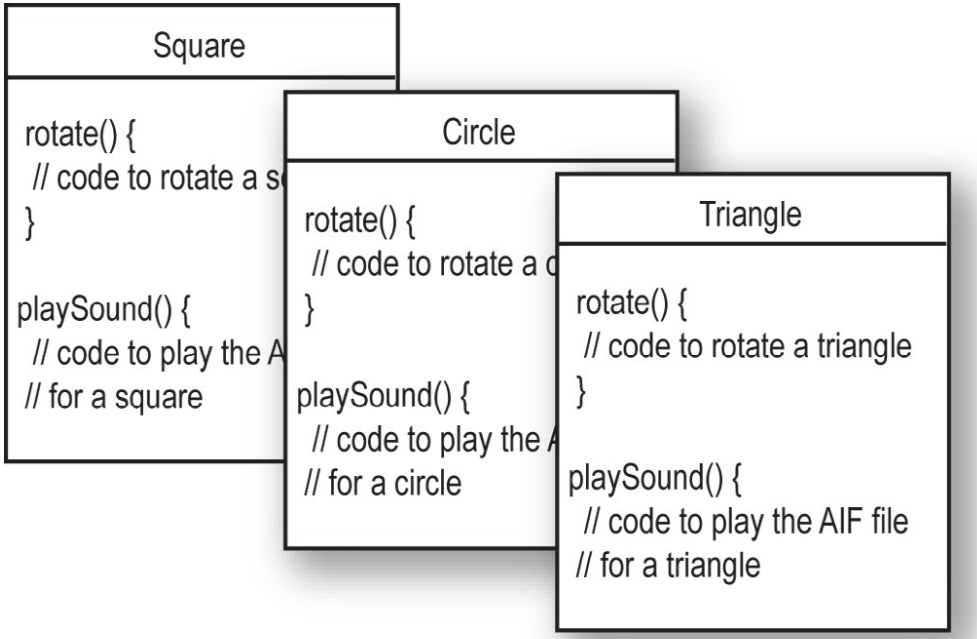
In una società di informatica un capo progetto mette in palio una sedia Aeron per chi sviluppa per primo un programma con determinate specifiche.

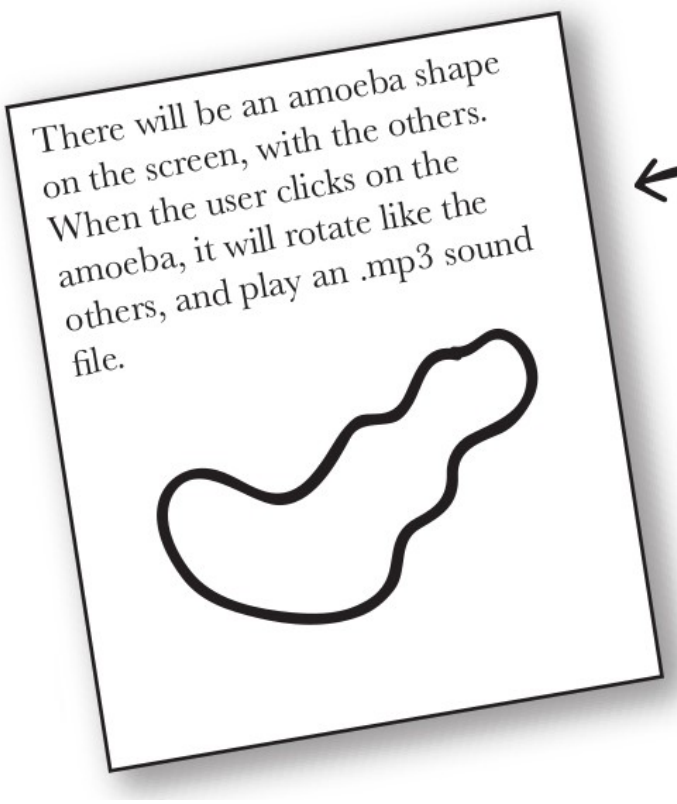
Alla gara partecipano Laura e Brad che seguono due approcci differenti: il primo procedurale, il secondo a oggetti.

Laura:

```
rotate(shapeNum) {
  // make the shape rotate 360°
}
playSound(shapeNum) {
  // use shapeNum to lookup which
  // AIF sound to play, and play it
}
```

Brad:






what got added to the spec

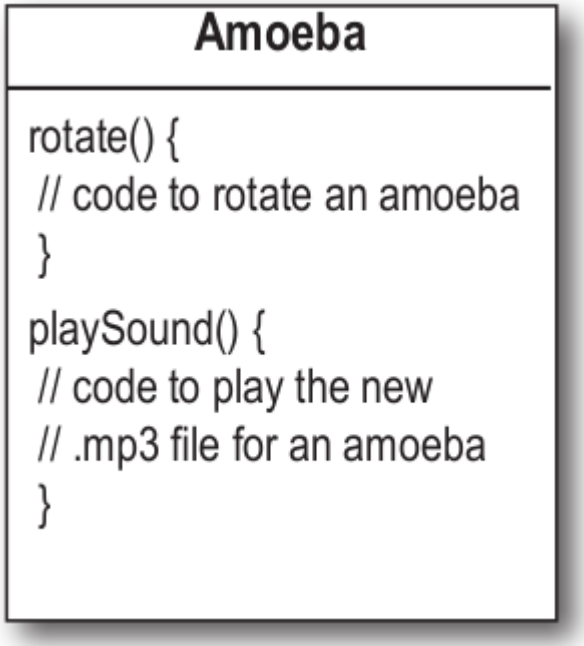
Laura è stato più veloce, ma non ha ancora vinto, perché come spesso accade c'è un cambiamento  nelle specifiche, quindi il programma è ancora aperto.

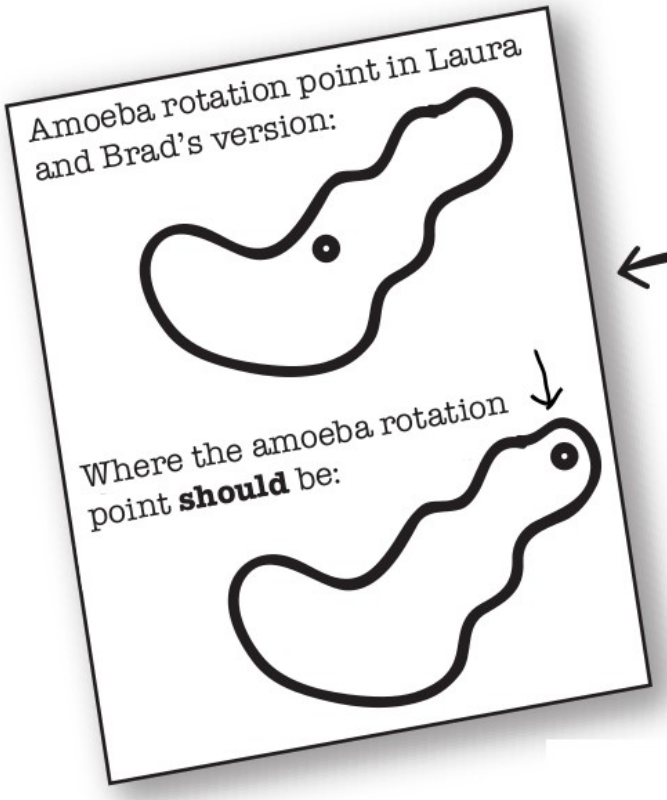
Tra le forme da gestire hanno aggiunto un'ameba.

Laura:

```
playSound(shapeNum) {
  // if the shape is not an amoeba,
    // use shapeNum to lookup which
    // AIF sound to play, and play it
  // else
    // play amoeba .mp3 sound
}
```

Brad:



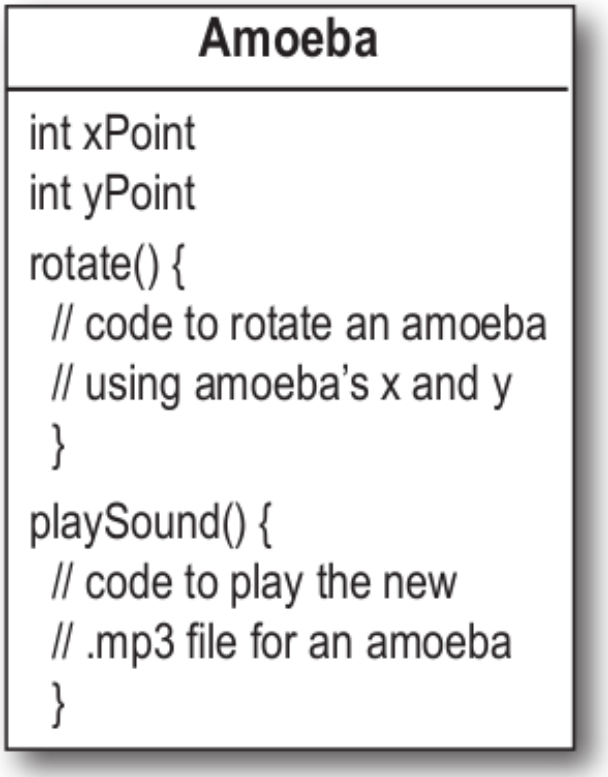


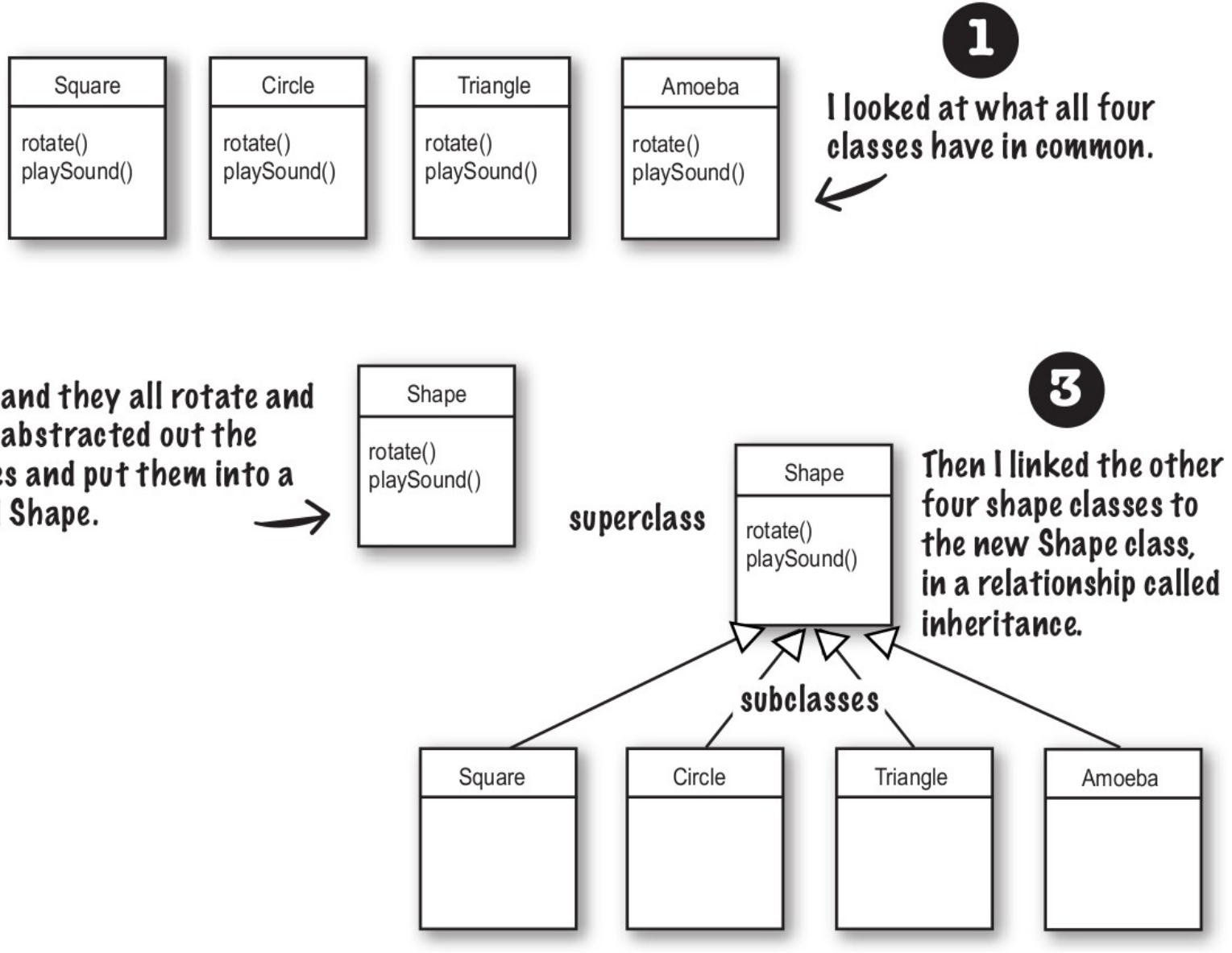

What the spec conveniently forgot to mention

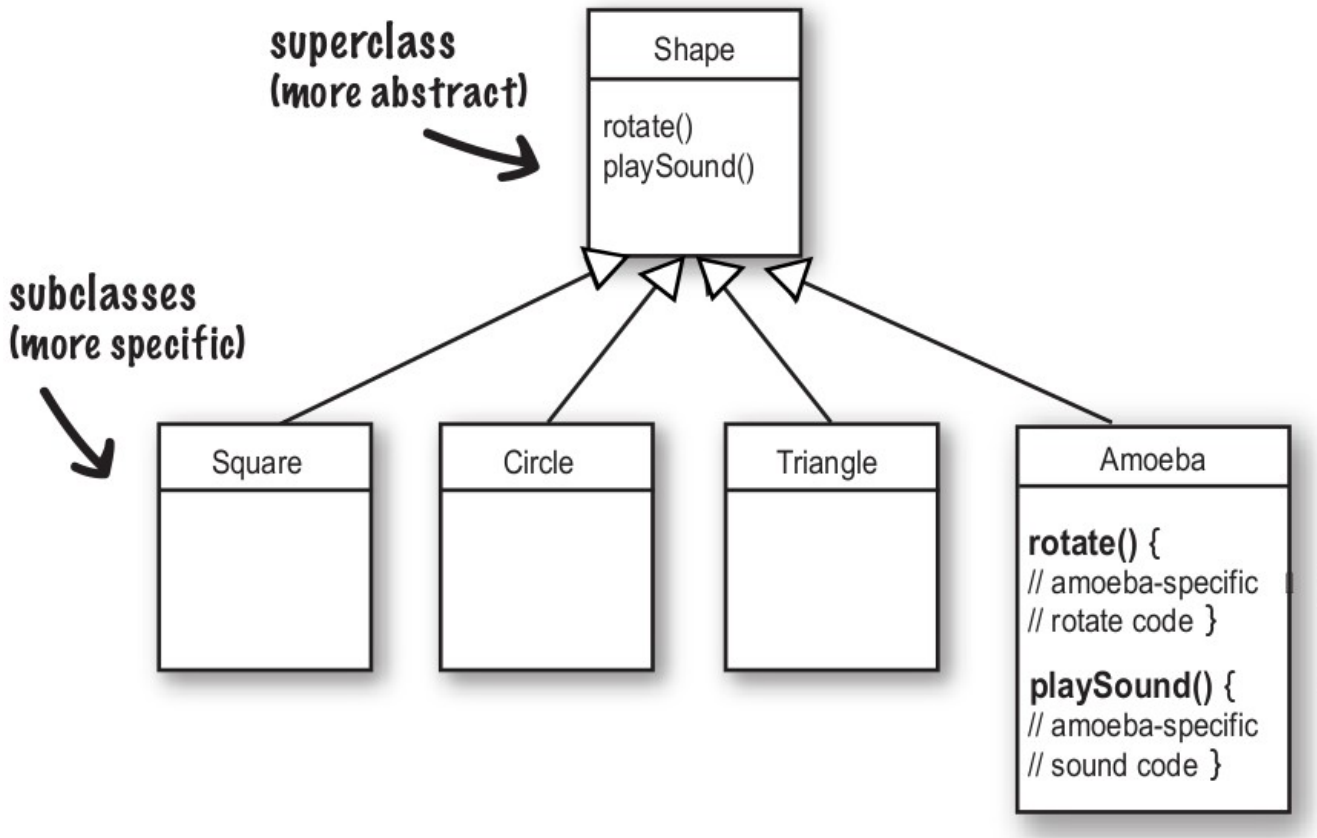
Laura:

```
rotate(shapeNum, xPt, yPt) {
  // if the shape is not an amoeba,
    // calculate the center point
    // based on a rectangle,
    // then rotate
  // else
    // use the xPt and yPt as
    // the rotation point offset
    // and then rotate
}
```

Brad:



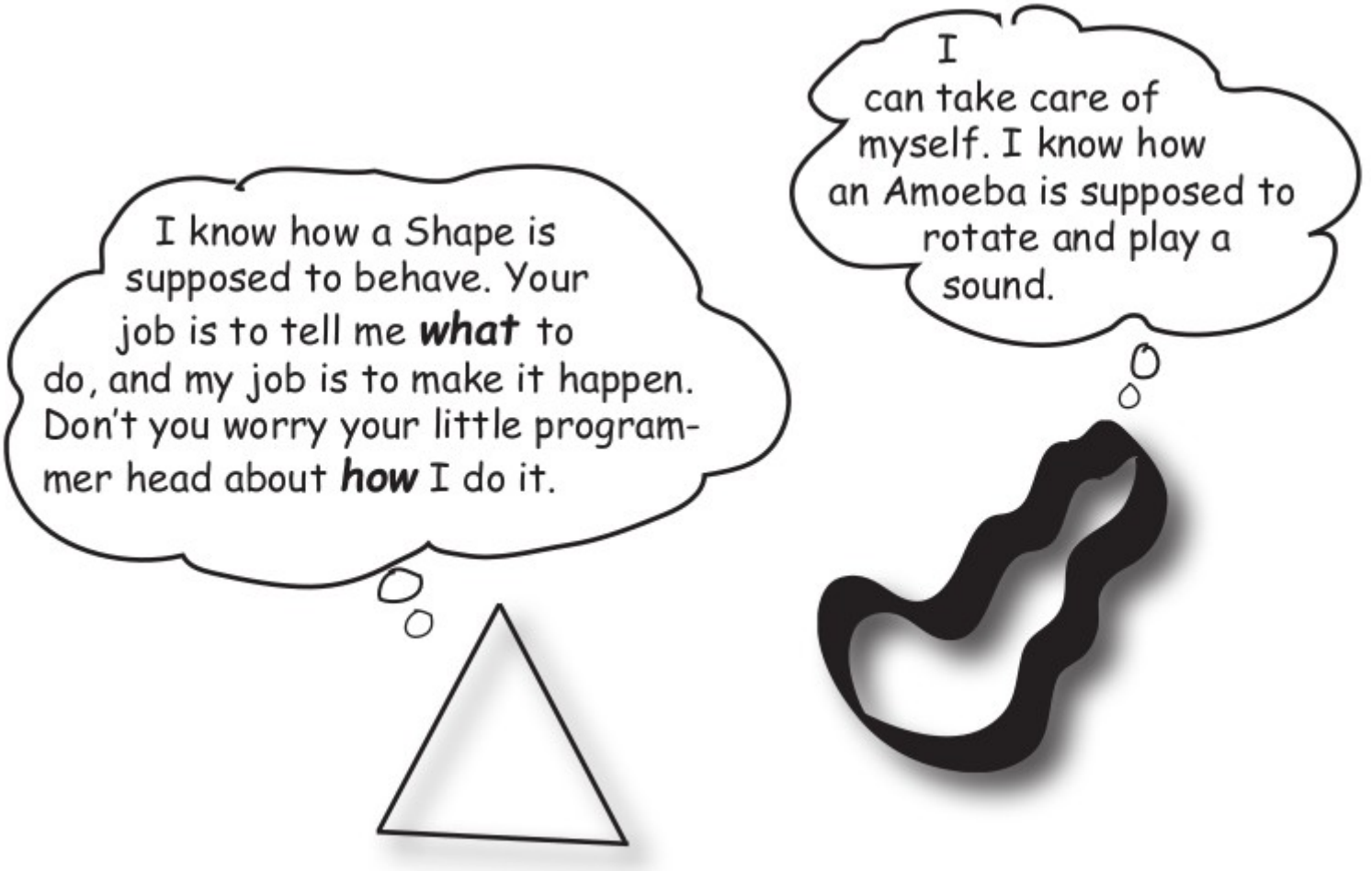

Square
rotate()
playSound()
Circle
rotate()
playSound()
Triangle
rotate()
playSound()
Amoeba
rotate()
playSound()
1
I looked at what all four classes have in common.
2
They're Shapes, and they all rotate and playSound. So I abstracted out the common features and put them into a new class called Shape.
Shape
rotate()
playSound()
3
Then I linked the other four shape classes to the new Shape class, in a relationship called inheritance.
superclass
Shape
rotate()
playSound()
subclasses
Square
Circle
Triangle
Amoeba




4

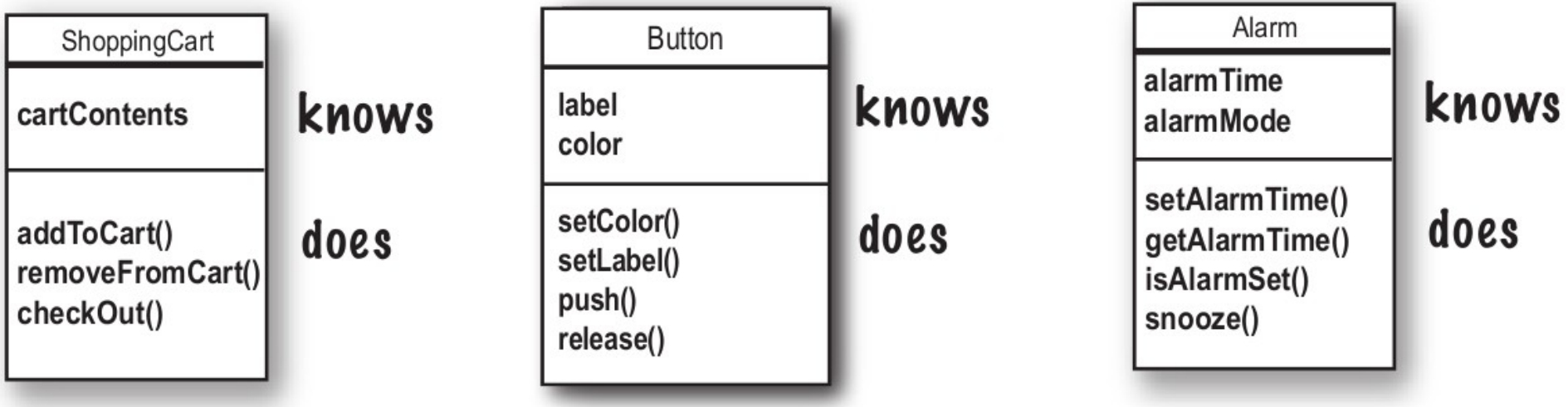
I made the Amoeba class override the rotate() and playSound() methods of the superclass Shape.

Overriding just means that a subclass redefines one of its inherited methods when it needs to change or extend the behavior of that method.


I know how a Shape is supposed to behave. Your job is to tell me **what** to do, and my job is to make it happen. Don't you worry your little programmer head about **how** I do it.
I can take care of myself. I know how an Amoeba is supposed to rotate and play a sound.

When you design a class, think about the objects that will be created from that class type. Think about:

- things the object **knows**
- things the object **does**



Things an object *knows* about itself are called

- instance variables

Things an object can *do* are called

- methods

**instance variables**
(state)

**methods**
(behavior)


knows

does

## Sharpen your pencil

Fill in what a television object might need to know and do.

| Television |
| --- |
| instance variables |
| methods |

# BE the compiler

**A**

```
class TapeDeck {

  boolean canRecord = false;

  void playTape() {
    System.out.println("tape playing");
  }

  void recordTape() {
    System.out.println("tape recording");
  }
}
```

```
class TapeDeckTestDrive {
  public static void main(String [] args) {

    t.canRecord = true;
    t.playTape();

    if (t.canRecord == true) {
      t.recordTape();
    }
  }
}
```

# BE the compiler

Manca l'istanza: TapeDeck t = new TapeDeck();

**A**

```
class TapeDeck {

  boolean canRecord = false;

  void playTape() {
    System.out.println("tape playing");
  }

  void recordTape() {
    System.out.println("tape recording");
  }
}
```

```
class TapeDeckTestDrive {
  public static void main(String [] args) {

    t.canRecord = true;
    t.playTape();

    if (t.canRecord == true) {
      t.recordTape();
    }
  }
}
```

# BE the compiler

```
class DVDPlayer {

  boolean canRecord = false;

  void recordDVD() {
    System.out.println("DVD recording");
  }
}
```

```
class DVDPlayerTestDrive {
  public static void main(String [] args) {

    DVDPlayer d = new DVDPlayer();
    d.canRecord = true;
    d.playDVD();

    if (d.canRecord == true) {
      d.recordDVD();
    }
  }
}
```

# BE the compiler

Il metodo playDVD() non esiste.

```
class DVDPlayer {

  boolean canRecord = false;

  void recordDVD() {
    System.out.println("DVD recording");
  }
}
```

```
class DVDPlayerTestDrive {
  public static void main(String [] args) {

    DVDPlayer d = new DVDPlayer();
    d.canRecord = true;
    d.playDVD();

    if (d.canRecord == true) {
      d.recordDVD();
    }
  }
}
```

# Who Am I?

**Tonight's attendees:**

**Class** **Method** **Object** **Instance variable**

| | |
|---|---|
| I am compiled from a .java file. | class |
| My instance variable values can be different from my buddy's values. | |
| I behave like a template. | |
| I like to do stuff. | |
| I can have many methods. | |
| I represent "state." | |
| I have behaviors. | |
| I am located in objects. | |

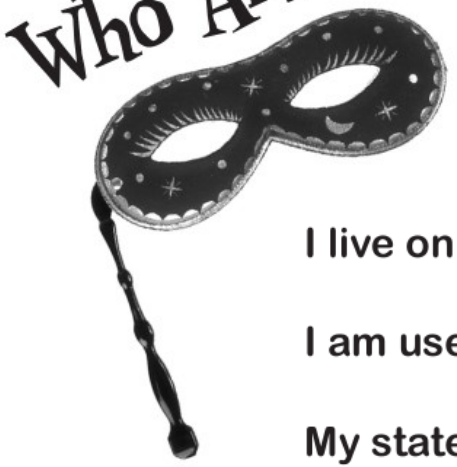
# Who Am I?

**Tonight's attendees:**

**Class** **Method** **Object** **Instance variable**

| | |
|---|---|
| **I am compiled from a .java file.** | class |
| **My instance variable values can be different from my buddy's values.** | object |
| **I behave like a template.** | class |
| **I like to do stuff.** | object, method |
| **I can have many methods.** | class, object |
| **I represent "state."** | Instance variable |
| **I have behaviors.** | object, class |
| **I am located in objects.** | method, instance variable |

# Who Am I?



**Tonight's attendees:**

**Class Method Object Instance variable**

I live on the heap. _____

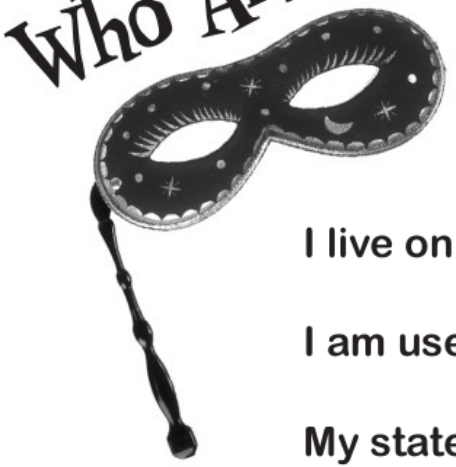
I am used to create object instances. _____

My state can change. _____

I declare methods. _____

I can change at runtime. _____

# Who Am I?



**Tonight's attendees:**

**Class Method Object Instance variable**

| | |
|---|---|
| **I live on the heap.** | object |
| **I am used to create object instances.** | class |
| **My state can change.** | object, instance variable |
| **I declare methods.** | class |
| **I can change at runtime.** | object, instance variable |

1 Declare an int array variable. An array variable is a remote control to an array object.

```
int[] nums;
```

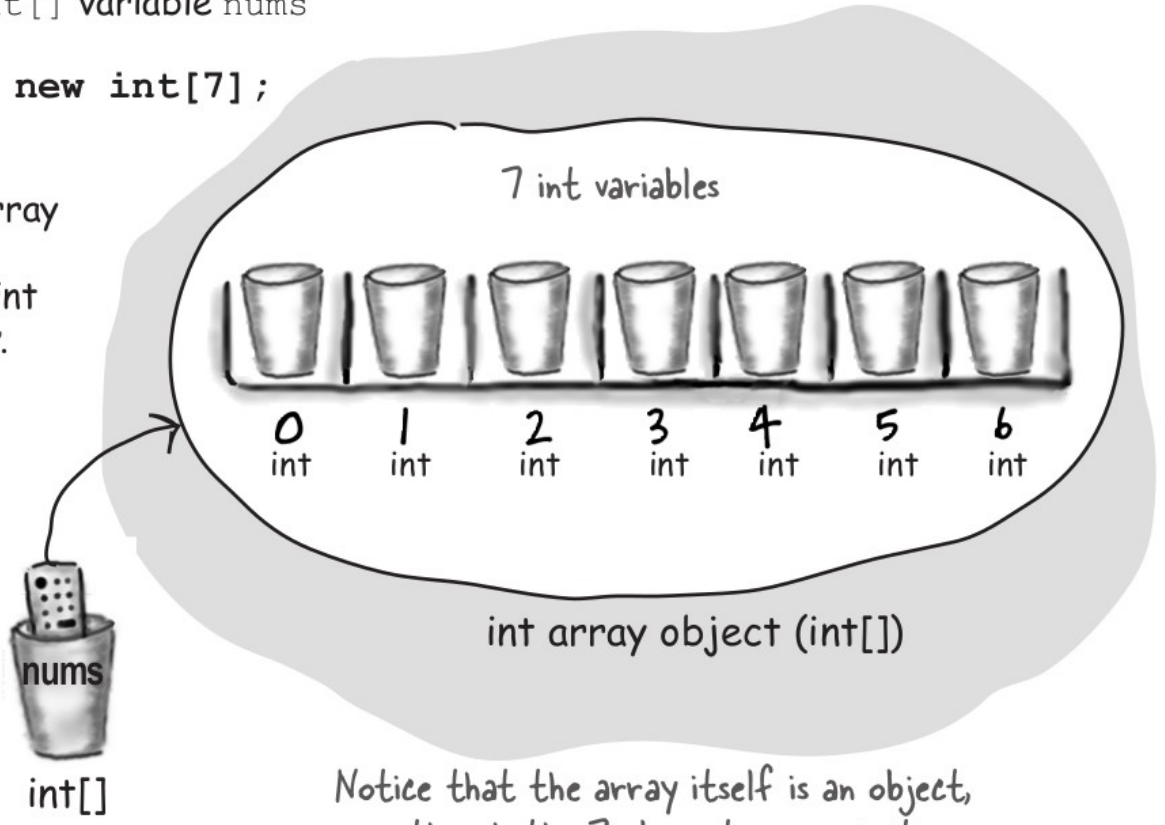
2 Create a new int array with a length of 7, and assign it to the previously declared `int[]` variable `nums`

```
nums = new int[7];
```

3 Give each element in the array some int value. Remember, elements in an int *array* are just int *variables*.

7 int variables

```
nums[0] = 6;
nums[1] = 19;
nums[2] = 44;
nums[3] = 42;
nums[4] = 10;
nums[5] = 20;
nums[6] = 1;
```




Arrays are always objects, whether they're declared to hold primitives or object references.

Notice that the array itself is an object, even though the 7 elements are primitives.

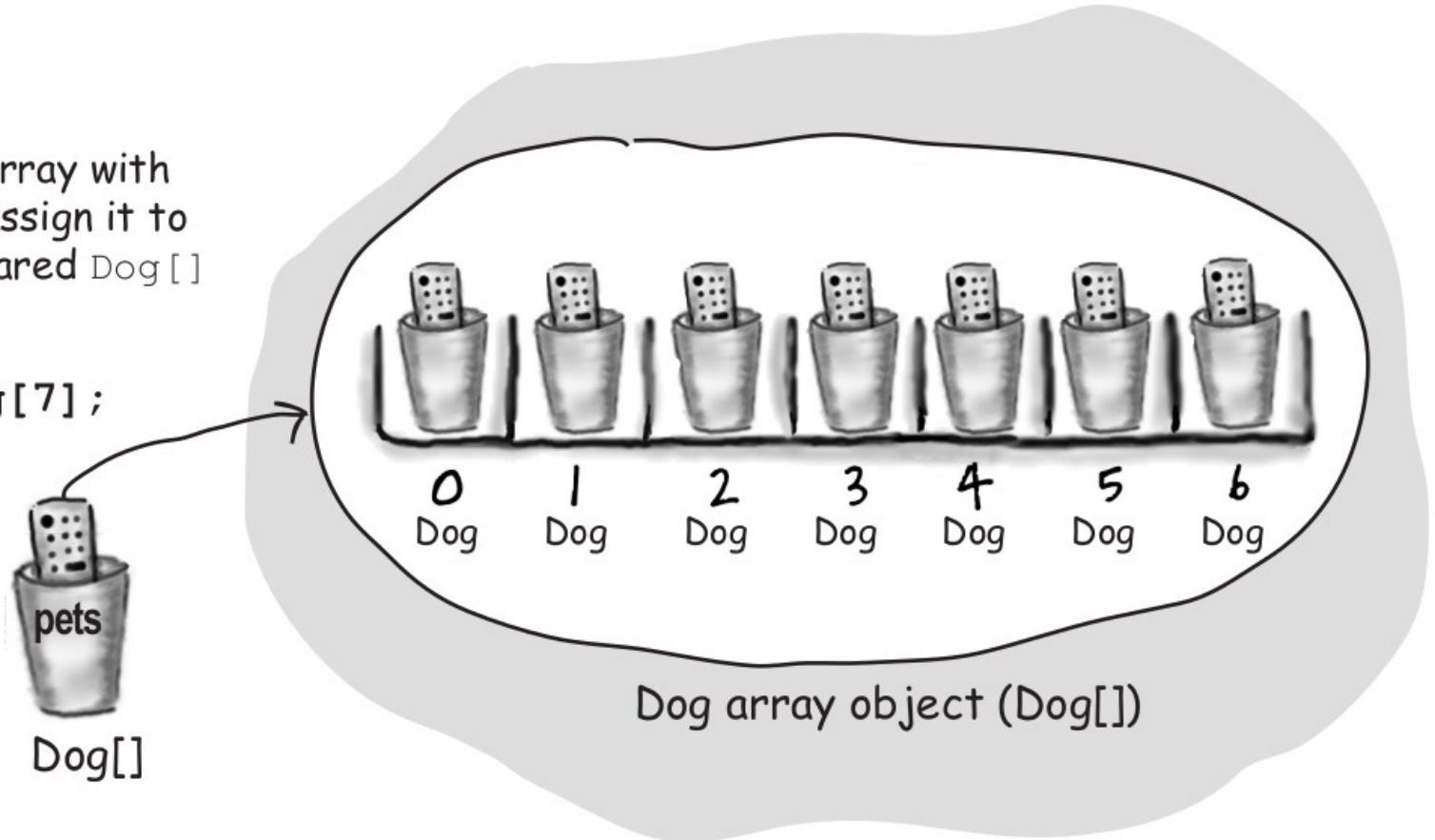
# Make an array of Dogs

1. Declare a Dog array variable

```
Dog[] pets;
```

2. Create a new Dog array with a length of 7, and assign it to the previously declared `Dog[]` variable `pets`

```
pets = new Dog[7];
```

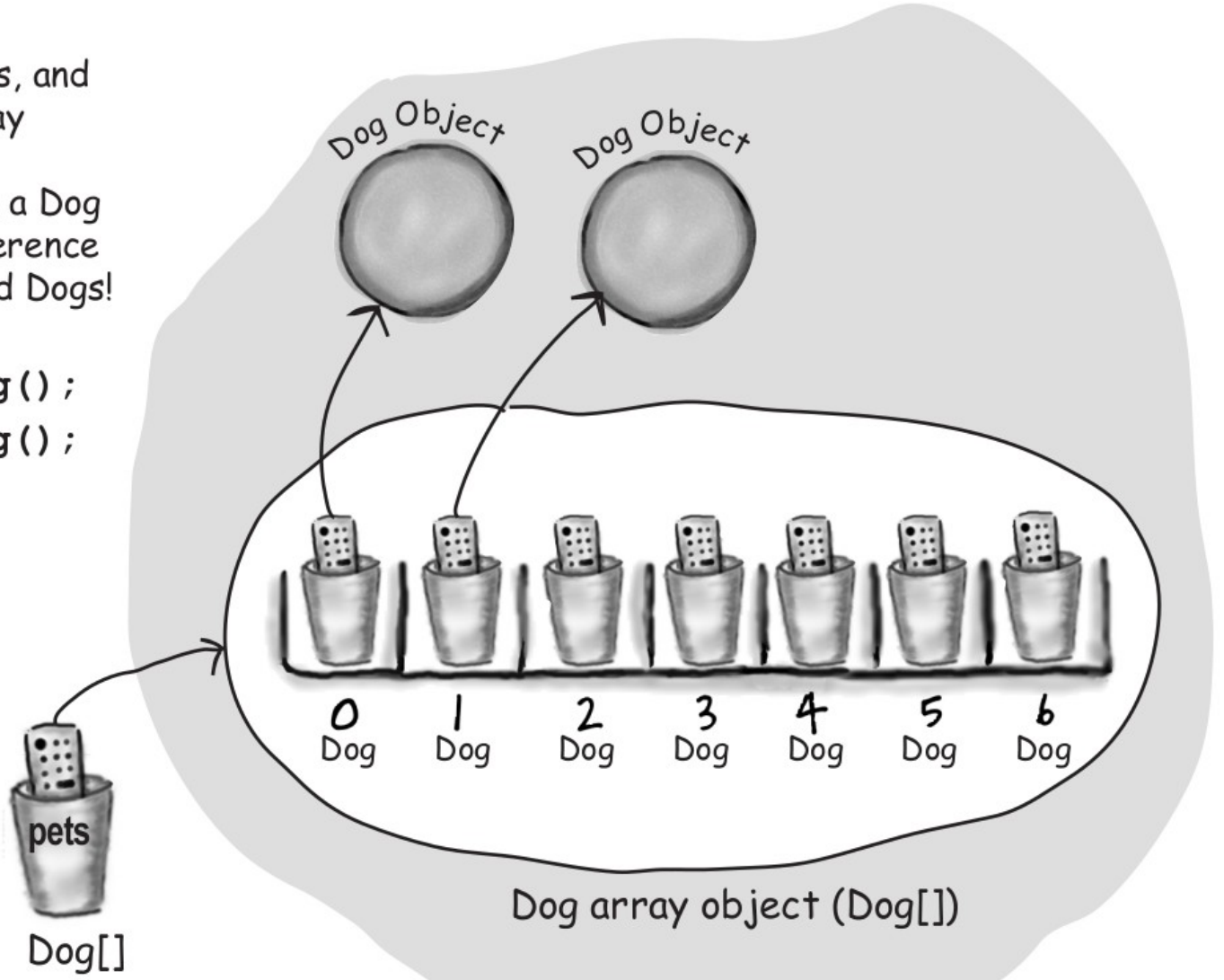


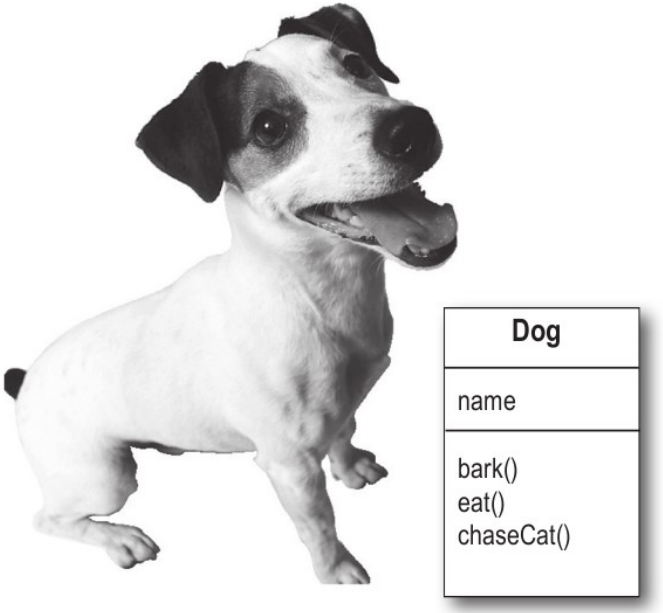

## What's missing?

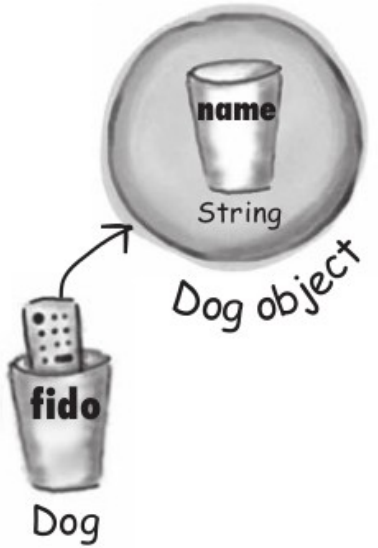
**Dogs! We have an array of Dog *references*, but no actual Dog *objects*!**

3 Create new Dog objects, and assign them to the array elements.
Remember, elements in a Dog *array* are just Dog reference *variables*. We still need Dogs!

```
pets[0] = new Dog();
pets[1] = new Dog();
```






```
Dog fido = new Dog();
fido.name = "Fido";
fido.bark();
fido.chaseCat();
```

```
Dog[] myDogs = new Dog[3];
myDogs[0] = new Dog();
myDogs[0].name = "Fido";
myDogs[0].bark();
```

# BE the compiler

```
class Books {
  String title;
  String author;
}
```

```
class BooksTestDrive {
  public static void main(String [] args) {

    Books [] myBooks = new Books[3];
    int x = 0;
    myBooks[0].title = "The Grapes of Java";
    myBooks[1].title = "The Java Gatsby";
    myBooks[2].title = "The Java Cookbook";
    myBooks[0].author = "bob";
    myBooks[1].author = "sue";
    myBooks[2].author = "ian";

    while (x < 3) {
      System.out.print(myBooks[x].title);
      System.out.print(" by ");
      System.out.println(myBooks[x].author);
      x = x + 1;
    }
  }
}
```

# BE the compiler

```
class Books {
  String title;
  String author;
}
```

Mancano gli oggetti!

myBooks[0] = new Books();

myBooks[1] = new Books();

myBooks[2] = new Books();

```
class BooksTestDrive {
  public static void main(String [] args) {

    Books [] myBooks = new Books[3];
    int x = 0;
    myBooks[0].title = "The Grapes of Java";
    myBooks[1].title = "The Java Gatsby";
    myBooks[2].title = "The Java Cookbook";
    myBooks[0].author = "bob";
    myBooks[1].author = "sue";
    myBooks[2].author = "ian";

    while (x < 3) {
      System.out.print(myBooks[x].title);
      System.out.print(" by ");
      System.out.println(myBooks[x].author);
      x = x + 1;
    }
  }
}
```

# BE the compiler

```
class Hobbits {

  String name;

  public static void main(String [] args) {

    Hobbits [] h = new Hobbits[3];
    int z = 0;

    while (z < 4) {
      z = z + 1;
      h[z] = new Hobbits();
      h[z].name = “bilbo”;
      if (z == 1) {
        h[z].name = “frodo”;
      }
      if (z == 2) {
        h[z].name = “sam”;
      }
      System.out.print(h[z].name + “ is a “);
      System.out.println(“good Hobbit name”);
    }
  }
}
```

# BE the compiler

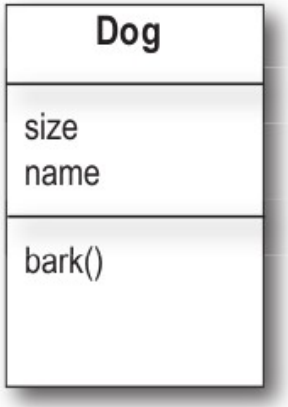
h[3] non esiste perché l'array ha solo 3 elementi, e parte da 0

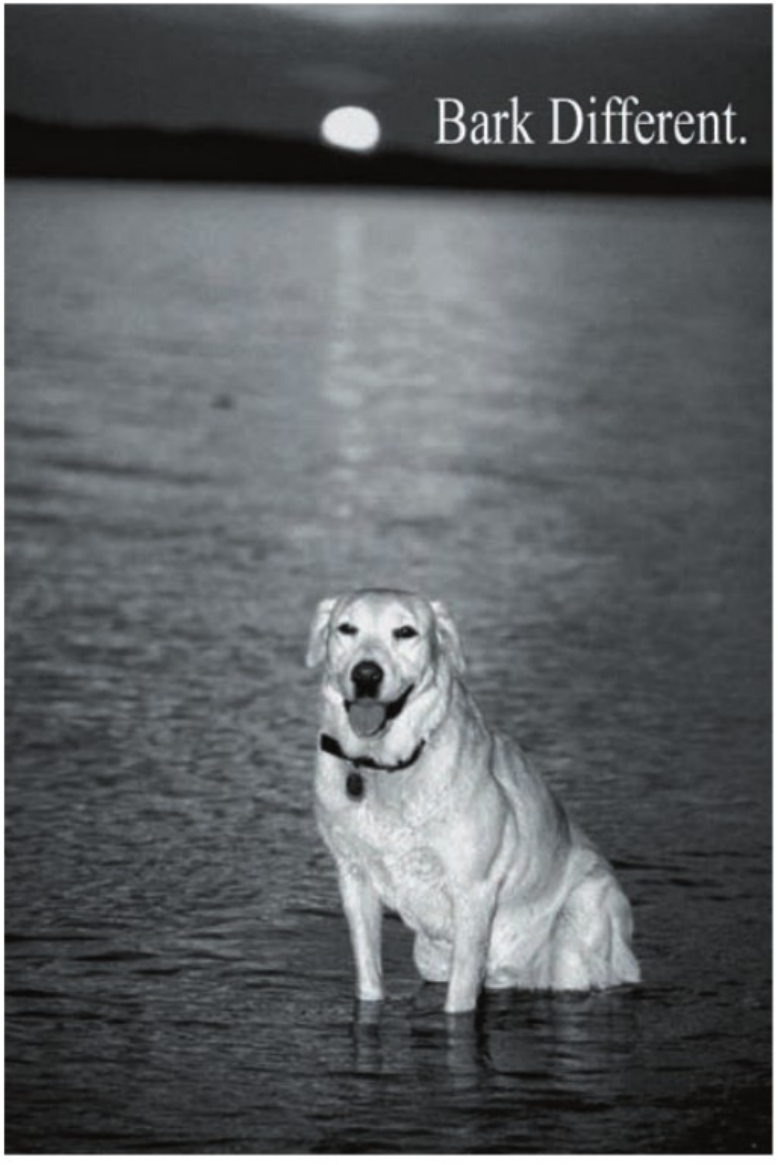
```
class Hobbits {

  String name;

  public static void main(String [] args) {

    Hobbits [] h = new Hobbits[3];
    int z = 0;

    while (z < 4) {
      z = z + 1;
      h[z] = new Hobbits();
      h[z].name = "bilbo";
      if (z == 1) {
        h[z].name = "frodo";
      }
      if (z == 2) {
        h[z].name = "sam";
      }
      System.out.print(h[z].name + " is a ");
      System.out.println("good Hobbit name");
    }
  }
}
```


Let's keep those little variables private, OK?

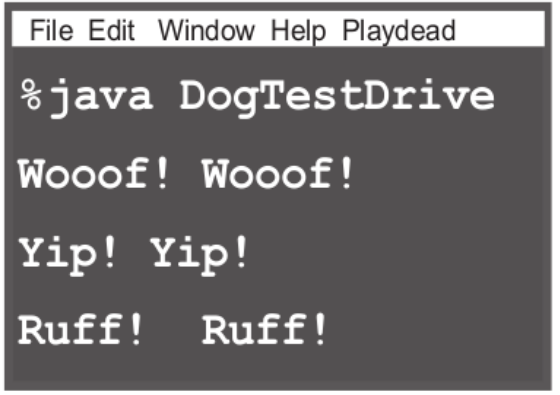
# The size affects the bark

```
class Dog {
  int size;
  String name;

  void bark() {
    if (size > 60) {
       System.out.println("Wooof! Wooof!");
    } else if (size > 14) {
       System.out.println("Ruff!  Ruff!");
    } else {
       System.out.println("Yip! Yip!");
    }
  }
}
```



```
class DogTestDrive {

   public static void main (String[] args) {
     Dog one = new Dog();
     one.size = 70;
     Dog two = new Dog();
     two.size = 8;
     Dog three = new Dog();
     three.size = 35;

     one.bark();
     two.bark();
     three.bark();
   }
}
```



```
Dog d = new Dog();
d.bark(3);
```

argument

1 Call the bark method on the Dog reference, and pass in the value 3 (as the argument to the method).

2 The bits representing the int value 3 are delivered into the bark method.

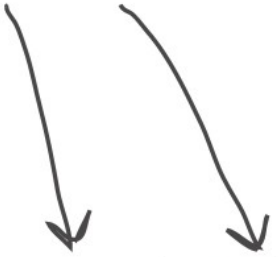

3 The bits land in the numOfBarks parameter (an int-sized variable).

```
void bark(int numOfBarks) {
  while (numOfBarks > 0) {
    System.out.println("ruff");
    numOfBarks = numOfBarks - 1;
  }
}
```

4 Use the numOfBarks parameter as a variable in the method code.

## Calling a two-parameter method, and sending it two arguments.

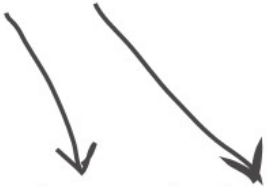
```
void go() {
   TestStuff t = new TestStuff();
   t.takeTwo(12, 34);
}

void takeTwo(int x, int y) {
   int z = x + y;
   System.out.println("Total is " + z);
}
```



The arguments you pass land in the same order you passed them. First argument lands in the first parameter, second argument in the second parameter, and so on.

## You can pass variables into a method, as long as the variable type matches the parameter type.

```
void go() {
    int foo = 7;
    int bar = 3;
    t.takeTwo(foo, bar);
}

void takeTwo(int x, int y) {
    int z = x + y;
    System.out.println("Total is " + z);
}
```



The values of foo and bar land in the x and y parameters. So now the bits in x are identical to the bits in foo (the bit pattern for the integer '7') and the bits in y are identical to the bits in bar.

What's the value of z? It's the same result you'd get if you added foo + bar at the time you passed them into the takeTwo method

Java is pass-by-value.

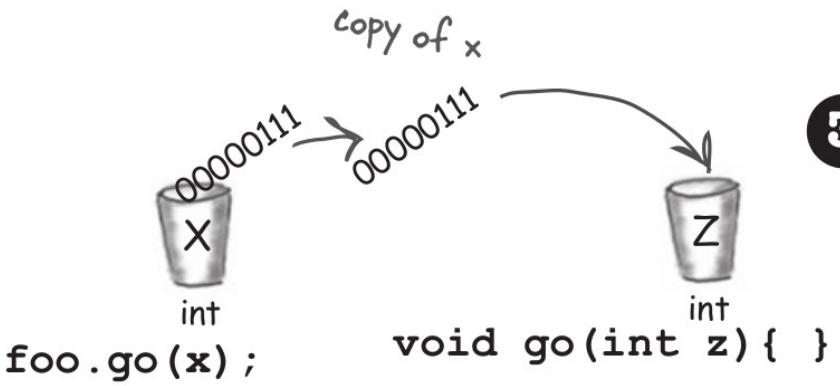
That means pass-by-copy.

```
int x = 7;
```

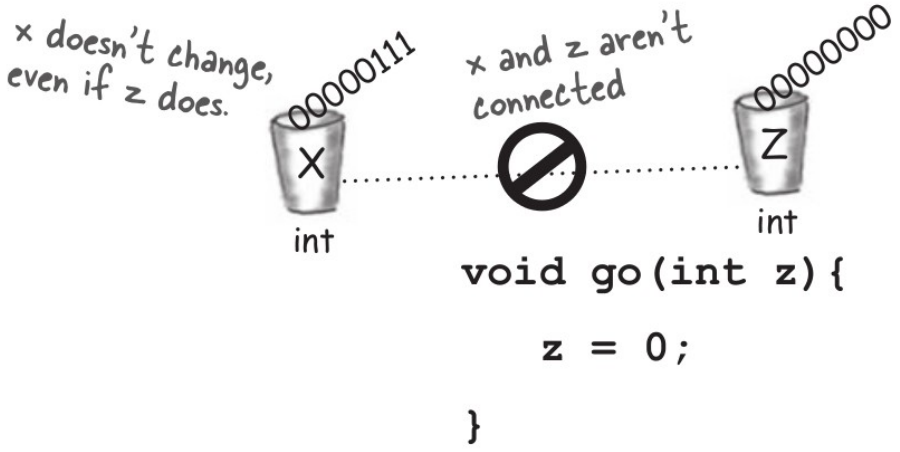

1. Declare an int variable and assign it the value '7'. The bit pattern for 7 goes into the variable named x.

```
void go(int z){ }
```


2. Declare a method with an int parameter named z.

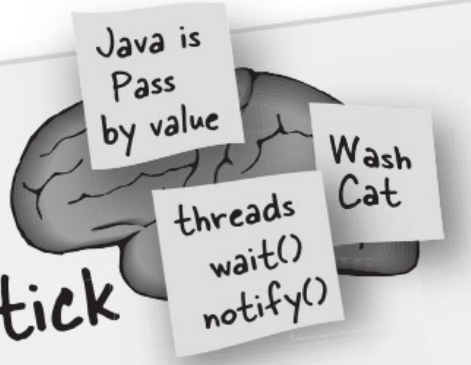



**3** Call the go() method, passing the variable x as the argument. The bits in x are copied, and the copy lands in z.




**4** Change the value of z inside the method. The value of x doesn't change! The argument passed to the z parameter was only a copy of x.

The method can't change the bits that were in the calling variable x.




# Make it Stick

*Roses are red,*
*this poem is choppy,*
*passing by value*
*is passing by copy.*

Oh, like you can do better? Try it. Replace our dumb second line with your own. Better yet, replace the whole thing with your own words and you'll *never* forget it.


Jen says you're
well-encapsulated...

```
theCat.height = 27;

theCat.height = 0;
```

yikes! We can't let this happen!

By forcing everybody to call a setter method, we can protect the cat from unacceptable size changes.

```
public void setHeight(int ht) {
    if (ht > 9) {
        height = ht;
    }
}
```

We put in checks to guarantee a minimum cat height.


RW
DVD+R DL
DVD
MULTI
RECORDER
COMPACT
disc
ReWritable
Ultra Speed


I always keep my variables private. If you want to see them, you have to talk to my methods.
private

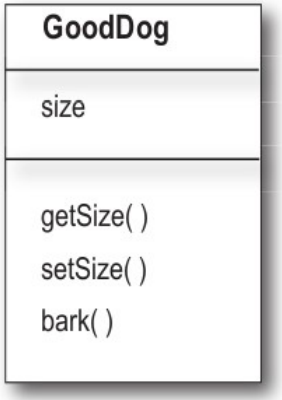
# Encapsulating the GoodDog class

Make the instance variable private.

Make the getter and setter methods public.

Even though the methods don't really add new functionality, the cool thing is that you can change your mind later. you can come back and make a method safer, faster, better.

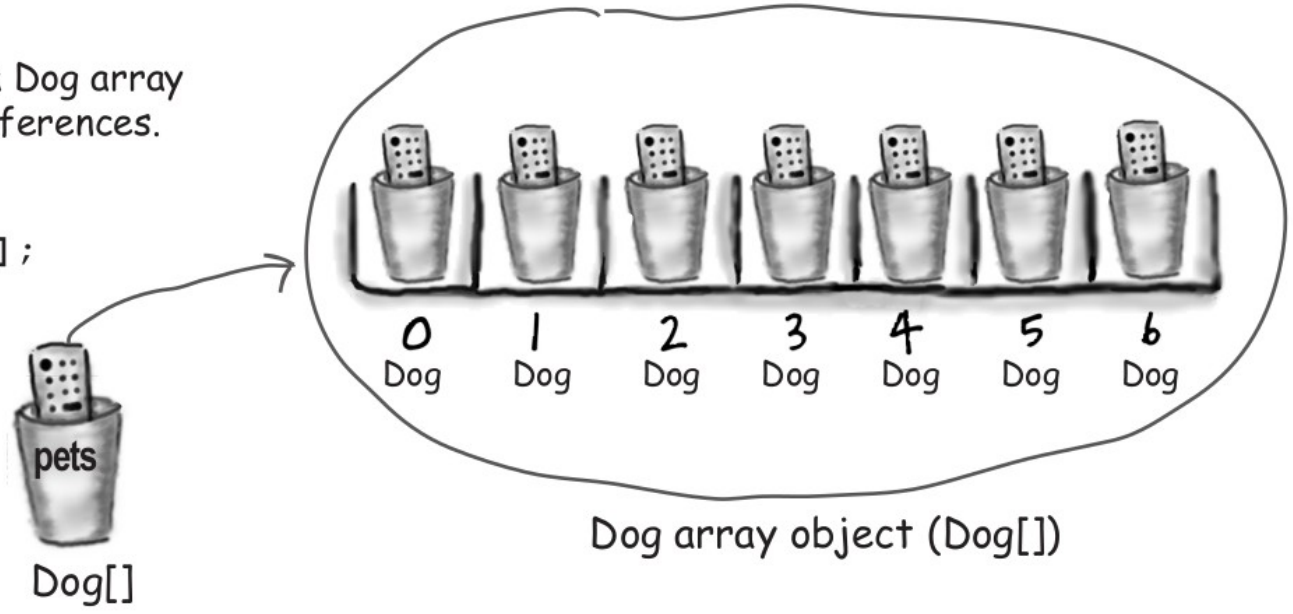
```
class GoodDog {

  private int size;

  public int getSize() {
    return size;
  }

  public void setSize(int s) {
    size = s;
  }

  void bark() {
    if (size > 60) {
      System.out.println("Wooof! Wooof!");
    } else if (size > 14) {
      System.out.println("Ruff!  Ruff!");
    } else {
      System.out.println("Yip! Yip!");
    }
  }
}
```



# How do objects in an array behave?

1. Declare and create a Dog array to hold seven Dog references.

```
Dog[] pets;
pets = new Dog[7];
```

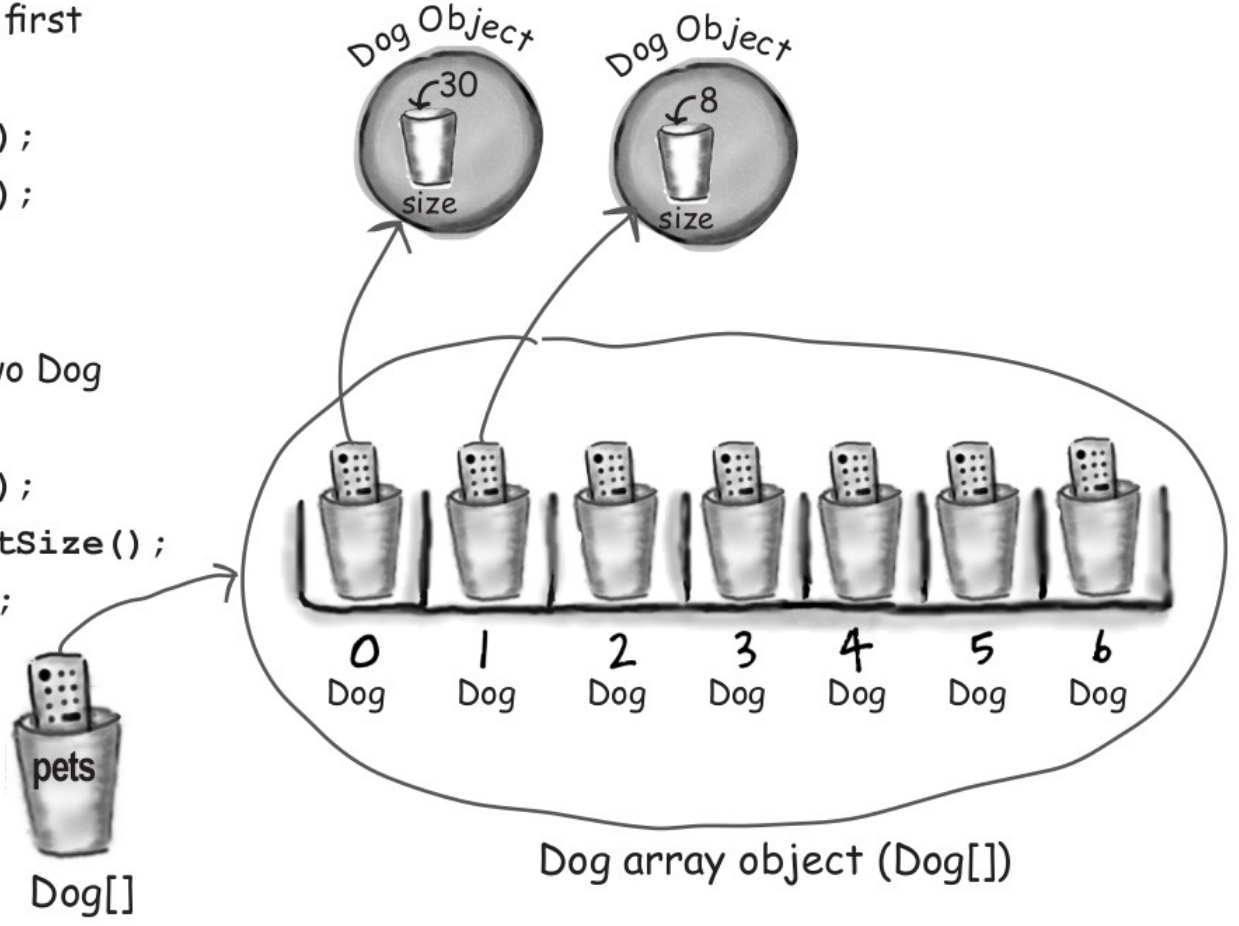


2 Create two new Dog objects, and assign them to the first two array elements.

```
pets[0] = new Dog();
pets[1] = new Dog();
```

3 Call methods on the two Dog objects.

```
pets[0].setSize(30);
int x = pets[0].getSize();
pets[1].setSize(8);
```



```
class PoorDog {
  private int size;
  private String name;

  public int getSize() {
    return size;
  }

  public String getName() {
    return name;
  }
}
```

Declare two instance variables, but don't assign a value

What will these return??

```
public class PoorDogTestDrive {
  public static void main(String[] args) {
    PoorDog one = new PoorDog();
    System.out.println("Dog size is " + one.getSize());
    System.out.println("Dog name is " + one.getName());
  }
}
```

What do you think? Will this even compile?

```
File Edit Window Help CallVet
% java PoorDogTestDrive
Dog size is 0
Dog name is null
```

**Instance variables always get a default value. If you don't explicitly assign a value to an instance variable or you don't call a setter method, the instance variable still has a value!**

| | |
|---|---|
| integers | 0 |
| floating points | 0.0 |
| booleans | false |
| references | null |

# The difference between instance and local variables

**1** **Instance** variables are declared inside a class but not within a method.

```
class Horse {
  private double height = 15.2;
  private String breed;
  // more code...
}
```

**2** **Local** variables are declared within a method.

```
class AddThing {
  int a;
  int b = 12;

  public int add() {
    int total = a + b;
    return total;
  }
}
```

INSTANCE variables

LOCAL variable

**3** **Local** variables MUST be initialized before use!

```
class Foo {
  public void go() {
    int x;
    int z = x + 3;
  }
}
```

Won't compile!! You can declare x without a value, but as soon as you try to USE it, the compiler freaks out.

```
File Edit Window Help Yikes
% javac Foo.java
Foo.java:4: variable x might not have been initialized
      int z = x + 3;
1 error       ^
```

# BE the compiler


```
int calcArea(int height, int width) {
   return height * width;
}
```

```
int a = calcArea(7, 12);
short c = 7;
calcArea(c,15);
int d = calcArea(57);
calcArea(2,3);
long t = 42;
int f = calcArea(t,17);
int g = calcArea();
calcArea();
byte h = calcArea(4,20);
int j = calcArea(2,3,5);
```

# BE the compiler


```
int calcArea(int height, int width) {
   return height * width;
}
```

```
int a = calcArea(7, 12);
short c = 7;
calcArea(c,15);
int d = calcArea(57);
calcArea(2,3);
long t = 42;
int f = calcArea(t,17);
int g = calcArea();
calcArea();
byte h = calcArea(4,20);
int j = calcArea(2,3,5);
```

# BE the compiler

```
class Clock {
  String time;

  void setTime(String t) {
    time = t;
  }

  void getTime() {
    return time;
  }
}
```

```
class ClockTestDrive {
  public static void main(String [] args) {

    Clock c = new Clock();

    c.setTime("1245");
    String tod = c.getTime();
    System.out.println("time: " + tod);
  }
}
```

# BE the compiler

```
class Clock {
  String time;

  void setTime(String t) {
    time = t;
  }

  void getTime() {
    return time;
  }
}
```

Dovrebbe essere String

```
class ClockTestDrive {
  public static void main(String [] args) {

    Clock c = new Clock();

    c.setTime("1245");
    String tod = c.getTime();
    System.out.println("time: " + tod);
  }
}
```


**Tonight's attendees:**

**instance variable, argument, return, getter, setter, encapsulation, public, private, pass by value, method**

A class can have any number of these. _____

A method can have only one of these. _____

I prefer my instance variables private. _____

A method can have many of these. _____

I return something by definition. _____

I can have many arguments. _____

By definition, I take one argument. _____

These help create encapsulation. _____

I always fly solo. _____


**Tonight's attendees:**

**instance variable, argument, return, getter, setter, encapsulation, public, private, pass by value, method**

A class can have any number of these. instance variable, getter, setter, method

A method can have only one of these. return

I prefer my instance variables private. encapsulation

A method can have many of these. argument

I return something by definition. getter

I can have many arguments. method

By definition, I take one argument. setter

These help create encapsulation. getter, setter, public, private

I always fly solo. return